

历年案例分析主题解题思路与强考模块

资料范围

本文件基于当前工作目录下 金考典/历年真题/案例分析 中的 PDF 归纳：

- 2020-11.pdf
- 2021-11.pdf
- 2022-11.pdf
- 2024-05.pdf
- 2024-11.pdf
- 2025-05.pdf
- 2025-11.pdf

说明：2025-11.pdf 当前只有回忆要点，完整题面暂缺，所以只用于趋势判断。

总体结论

案例分析的核心不是背某一道题，而是识别题目背后的固定模板。

近几年强考模块可以概括为五条主线：

1. 软件架构设计与评估：质量属性、效用树、质量属性场景、架构风格选择。
2. 数据管理与缓存：Redis、MySQL、反规范化、数据一致性、分片、布隆过滤器、MongoDB、HDFS。
3. 系统分析与建模：UML 用例图、顺序图、协作图，DFD，E-R 图，数据字典。
4. Web、云、边缘、物联网与分布式架构：三层架构、SSM、RESTful、TCP/UDP、MQTT、云平台、边缘计算。
5. 新技术场景套壳：大模型、知识图谱、端侧 AI、区块链、ROS、Elasticsearch、限界上下文。

考试趋势是：题目场景越来越新，但问法仍然很传统。看到大模型、知识图谱、区块链、端侧 AI，不要慌，通常仍在考“定义、架构图填空、优缺点、选型原因、质量属性、数据一致性、可靠性和性能权衡”。

历年主题地图

2020年11月

- 第1题：在线软件开发系统。
 - 主题：架构风格选择与质量属性。 - 考点：仓库风格与管道-过滤器风格对比，质量属性识别。 - 解题抓手：在线代码编辑、代码仓库、交互式共享数据更像仓库风格；响应时间是性能，攻击防御是安全性，故障切换是可用性，几天天修改是可修改性。
- 第2题：包裹信息管理。
 - 主题：关系数据库逻辑建模。 - 考点：逻辑数据模型设计、超类实体、派生属性。 - 解题抓手：从多个包裹单中抽公共字段形成超类；由其他字段计算得到的是派生属性。
- 第3题：评论交流平台数据管理。

- 主题：Redis 与数据库组合。 - 考点：Redis 数据类型、过期删除、内存淘汰。 - 解题抓手：计数用 String，排名用 ZSet，共同好友用 Set，结构化对象用 Hash，发布订阅用 Pub/Sub。

• 第4题：工业设备 Web 检测系统。

- 主题：Web 架构与非功能需求。 - 考点：性能、安全性、可用性分类，三层拓扑，SSM/MVC 工作流程。 - 解题抓手：时间、并发、吞吐归性能；XSS、审计归安全；7*24 和故障恢复归可用性。

2021年11月

• 第1题：机器学习应用开发平台。

- 主题：质量属性效用树与架构风格比较。 - 考点：解释器、管道-过滤器、隐式调用。 - 解题抓手：拖拽算法组件、灵活定义流程、新算法组件扩展，优先考虑解释器风格。

• 第2题：线上药品销售系统。

- 主题：MySQL 与 Redis 混合架构。 - 考点：反规范化、数据不一致、Redis ZSet、Redis 与 MySQL 同步。 - 解题抓手：查询商品要同时展示药品、供应商、库存，可用冗余字段/表合并提升查询性能；排名用 ZSet。

• 第3题：预约挂号管理系统。

- 主题：面向对象分析与 UML。 - 考点：用例图、顺序图、协作图、对象模型、动态模型、功能模型。 - 解题抓手：参与者看系统外部角色；顺序图看时间顺序；协作图看对象链接关系。

• 第4题：智能家居管理系统。

- 主题：云平台架构与通信协议。 - 考点：家庭网关 vs 云平台，架构图填空，TCP/UDP 对比。 - 解题抓手：要求可靠双向通信时优先 TCP；云平台有集中管理、弹性扩展、数据分析优势。

2022年11月

• 第1题：会员与促销管理系统。

- 主题：质量属性效用树与架构风格选择。 - 考点：面向对象 vs 解释器。 - 解题抓手：规则可配置、促销逻辑动态生效、性能不是首要目标，优先解释器风格。

• 第2题：煤矿安全预警系统。

- 主题：结构化分析与数据建模。 - 考点：DFD、E-R 图、数据字典、数据平衡原则。 - 解题抓手：DFD 处理名称来自功能列表；E-R 实体来自业务名词；父图和子图输入输出必须平衡。

• 第3题：分布式故障检测与诊断。

- 主题：可靠性与故障诊断。 - 考点：心跳检测、超时探测、监控器分类、定量故障诊断方法。 - 解题抓手：心跳是周期性报活，超时探测是请求超时判故障；故障归类按影响范围由核、分区、模块、系统逐层判断。

• 第4题：全国仓储货物管理系统。

- 主题：缓存集群与高性能数据访问。 - 考点：同步/异步更新、一致性哈希、缓存分片、布隆过滤器。 - 解题抓手：实时反馈小于1秒，适合缓存；节点变化频繁时，一致性哈希减少数据迁移；非法查询导致缓存击穿可用布隆过滤器拦截。

• 第5题：边缘计算智能门禁系统。

- 主题：边缘计算、协议选择、云边协同。 - 考点：MQTT、HTTP、Docker、边缘计算优势。 - 解题抓手：设备状态、轻量消息、低带宽场景用 MQTT；后台管理、接口调用用 HTTP；边缘计算强调低时延、少传输、本地自治和隐私。

2024年5月

- 第1题：软件架构设计与评估。

- 主题：微服务、SOA、质量属性。 - 考点：微服务优缺点、质量效用树、质量属性六元素。 - 解题抓手：微服务答“围绕业务能力拆分、独立部署、轻量通信、独立扩展”；六元素固定为刺激源、刺激、环境、制品、响应、响应度量。

- 第2-4题：UML 交互图。

- 主题：时序图与协作图。 - 考点：同步消息、异步消息、返回消息，时序图和协作图选取原则，组合片段。 - 解题抓手：强调时间先后用时序图；强调对象关系和链接结构用协作图。

- 第5题：MongoDB。

- 主题：非关系型数据库与空间数据。 - 考点：GeoJSON、空间索引、文档型数据库优势。 - 解题抓手：非结构化/半结构化数据、字段灵活、横向扩展、地理空间查询是 MongoDB 的高频关键词。

- 第6题：HDFS 冷热数据。

- 主题：大数据存储分层。 - 考点：热数据、温数据、冷数据，SSD/SAS/SATA。 - 解题抓手：访问频率越高越用高性能介质，访问频率越低越用低成本大容量介质。

2024年11月

- 第1题：云服务器故障检测。

- 主题：质量属性、可靠性、心跳机制。 - 考点：功能性/质量属性分类，ping/echo 与心跳对比。 - 解题抓手：主动探测是 ping/echo；被监控方周期报活是心跳；从资源利用率看心跳开销小、消息短、网络负载低。

- 第2题：Cache-Aside 架构。

- 主题：数据库缓存与一致性。 - 考点：读流程、写流程、并发读写不一致、解决方案。 - 解题抓手：读是先缓存，未命中查库并回填；写是先更新数据库，再删除缓存；解决方案包括延迟双删、删除失败重试、MQ/binlog、互斥锁、TTL、版本号。

- 第3题：机器人操作系统 ROS。

- 主题：嵌入式与机器人中间件。 - 考点：ROS 定义、ROS2 改进、话题/服务/动作/参数服务。 - 解题抓手：ROS 是机器人软件开发的元操作系统；ROS2 重点是 DDS、实时性、安全性、多平台。

- 第4题：Elasticsearch 商品推荐系统。

- 主题：搜索与前后端架构。 - 考点：Standard/Simple/Whitespace/Keyword 分词器，RESTful 前后端分离。 - 解题抓手：RESTful 答资源 URL、HTTP 动词、JSON 响应、前后端职责分离。

2025年5月

- 第1题：在线大模型训练平台。

- 主题：解释器风格与质量属性。 - 考点：质量属性识别、解释器架构组成、适用原因。 - 解题抓手：Python 代码编写、解析、动态执行、资源匹配，是解释器风格典型场景。

- 第2题：医药知识图谱智能问答。

- 主题：知识图谱、爬虫、异步 I/O。 - 考点：知识图谱分层架构、Scrapy 架构、异步 I/O、知识存储。 - 解题抓手：知识图谱通常按数据层、知识层、业务层等组织；知识存储可答图数据库/RDF 三元组，强调关系表达和图查询。

- 第3题：端侧 AI 与资源池。

- 主题：云端 AI、端侧 AI、资源池架构。 - 考点：资源抽象与异构计算、动态调度与能效优化、安全隔离，集中式/分布式/混合资源池。 - 解题抓手：端侧 AI

优势是低延迟、隐私保护、弱网可用、减少带宽；资源池题按“资源抽象、调度优化、安全隔离”三类归位。

- 第4题：Redis 主从与持久化。

- 主题：Redis 高可用与持久化。 - 考点：首次全量同步、增量同步、RDB、AOF。 -

解题抓手：全量同步是请求同步、生成 RDB、缓冲写命令、发送 RDB、发送增量命令；RDB 恢复快但可能丢数据，AOF 更完整但文件大、恢复慢。

- 第5题：区块链农产品检验流程。

- 主题：区块链与智能合约。 - 考点：区块链分层、上链流程、智能合约作用。 - 解题抓手：区块链数据层、网络层、共识层、激励层、合约层、应用层；智能合约自动执行、规则固化、可信协作和可追溯。

2025年11月

- 当前资料：题面暂缺，只有回忆要点。

- 必答方向：软件质量属性场景，AES-256 固定 16 字节数据加密，存储和传输。

- 选答方向：精简操作系统、智能操作系统、限界上下文、餐饮系统架构图、低质量网络传输。

- 复习判断：仍然延续“质量属性场景 + 新技术/架构概念 + 场景化方案”的模式。

历年题目答案要点

说明：以下答案按当前 PDF 中的“试题解析/标准答案”整理。2024

年以后部分资料属于回忆版，题图和选项不完整的位置，写成“答案要点”而不是唯一标准答案。

2020年11月答案

第1题：在线软件开发系统

问题1：

应采用仓库风格。

管道-过滤器与仓库风格比较表答案：

- (1) 文件或模型驱动机制，处理流程事先不确定，交互性强
- (2) 数据与处理紧耦合，以接口适配方式实现扩展
- (3) 中央系统构件体现系统状态，系统执行没有确定顺序，性能相对较低
- (4) 共享中心数据，加载共享数据，性能相对较高

问题2：

- (1) 安全性
- (2) 可修改性
- (3) g
- (4) i
- (5) f
- (6) j

第2题：包裹信息管理系统

问题1：

逻辑结构设计阶段任务：
确定数据模型；
将 E-R 图转换为指定数据模型；
确定完整性约束；
确定用户视图。

可设计的关系模式包括：

收件人信息，主键：电话号码。
寄件人信息，主键：电话号码或身份证号。
包裹单信息，主键：包裹单编号。
快递员信息，主键：员工编号。
邮局站点信息，主键：邮局编号。

问题2：

超类实体是将若干子实体共有属性抽象出来形成的新实体。
示例：人员信息(姓名，电话，单位名称，详细地址，邮政编码)。

问题3：

派生属性是可由其他属性计算得到的属性。
示例：快递包裹费用中的“总计”属性。

第3题：Redis 与数据库方案

问题1：

- (1) a
- (2) b
- (3) c、d
- (4) f
- (5) e

问题2：

定期删除采用周期性随机抽样，不可能扫描并删除全部过期 key；
惰性删除只有访问 key 时才检查是否过期。
如果某些过期 key 没被定期抽中，也长期没有被访问，就不会被删除，内存占用会持续升高。

任选三种内存淘汰机制即可：

volatile-lru：从设置过期时间的数据中淘汰最近最少使用的数据。
volatile-ttl：从设置过期时间的数据中淘汰将要过期的数据。
volatile-random：从设置过期时间的数据中随机淘汰。
volatile-lfu：从设置过期时间的数据中淘汰最低频使用数据。
allkeys-lru：从全部数据中淘汰最近最少使用的数据。
allkeys-lfu：从全部数据中淘汰最低频使用数据。
allkeys-random：从全部数据中随机淘汰。
noeviction：不淘汰，写入时报错。

第4题：工业设备 Web 检测系统

问题1：

性能：(b)、(a)
安全性：(d)、(f)
可用性：(c)、(e)

问题2：

- (1) a：Connection pool
- (2) c：Persistent Layer
- (3) d：Mybatis
- (4) k：Spring
- (5) j：Controller Layer
- (6) h：ViewLayer
- (7) i：Jsp

问题3：

采用标准的数据访问机制，可以屏蔽不同设备之间的数据交互差异，解决系统使用数据不一致问题，降低数据结构与应用系统的耦合度，减少系统维护工作量。

2021年11月答案

第1题：机器学习应用开发平台

问题1：

- (1) 性能
- (2) 可修改性
- (3) e：可用性
- (4) j：性能
- (5) h：安全性
- (6) i：可修改性

问题2：

更适合解释器架构风格。

比较要点：

解释器：流程定义灵活性高，算法组件扩展能力强，可通过自定义流程规则和解释引擎支持用户定义流程。
管道-过滤器：适合固定流程的数据处理，流程变化或新增能力时需要修改过滤器或重建流程，灵活性较低。
隐式调用：通过事件机制提升扩展性，灵活性强于管道-过滤器，但弱于用户可完全自定义规则的解释器。

第2题：线上药品销售系统

问题1：

常见反规范化技术：增加冗余列、增加派生列、重新组表、分割表。
本题查询商品信息时需要同时展示药品、供应商和库存信息，应采用增加冗余列，把供应商名称、当前库存数量等冗余到药品关系中，以减少多表连接。

问题2：

解决反规范化数据不一致的方法：
触发器数据同步；
应用程序数据同步；
物化视图。
也可答批处理维护、应用逻辑维护、触发器维护。

问题3：

热销药品排名使用 ZSet，即有序集合。

Redis 与 MySQL 同步方案：

先查缓存，查不到再查数据库并写入缓存；
更新时先更新数据库，再删除或更新缓存；
可用消息队列异步同步；
可通过数据库插件或 binlog 监听同步；
可通过触发器同步；
也可利用事务、中间件或访问控制位辅助保证一致性。

第3题：预约挂号管理系统

问题1：

- (1) 系统管理员

- (2) 患者
- (3) a：注册登录
- (4)-(8) c：账号管理，f：号源管理，h：预约管理，i：报表管理，j：信用管理
- (9)-(12) b：信息浏览，d：预约挂号，e：查询与取消预约，g：报告查询

问题2：

- (1) 预约人员/患者
- (2) 预约挂号请求
- (3) 显示医生可预约时段
- (4) 显示预约是否成功

顺序图与协作图区别：

顺序图强调对象交互的时间次序；
协作图强调对象之间的组织结构和协作关系。

问题3：

对象模型描述系统静态结构，是最基础、最核心的模型。
动态模型描述系统交互次序和状态变化。
功能模型描述系统的数据变换。

三者关系：

对象模型描述动态模型和功能模型所操作的数据结构；
动态模型描述对象模型的控制结构，并触发功能；
功能模型描述由对象操作和动态动作激活的数据处理功能。

第4题：智能家居管理系统

问题1：

网关管理：云平台可远程统一管理不同地点、多种设备，管理能力更强。
数据处理：数据上传云端后便于集中存储、分析和挖掘，也具备更好的容灾能力。
系统性能：云平台具备更强的计算和存储能力，通信与数据处理效率更高。

问题2：

- (1) h：用户终端
- (2) i：鸿蒙
- (3) f：云平台
- (4) d：数据库
- (5) e：家庭网关
- (6) c：驱动程序

问题3：

应采用 TCP 协议。
TCP 是面向连接、可靠的、基于字节流的传输层协议，通过三次握手、确认应答和重传机制保证通信可靠。
UDP 是无连接协议，丢包后不保证重传，不能满足可靠双向通信要求。

2022年11月答案

第1题：会员与促销管理系统

问题1：

- (1) 安全性
- (2) 可修改性
- (3) e
- (4) j
- (5) h

(6) k

问题2：

更适合解释器架构风格。

答案要点：

本系统强调折扣规则和促销逻辑灵活设置、设置后立即生效，并且性能不是首要目标。
解释器风格可把折扣规则和用户筛选条件封装为规则，由解释器动态解释执行，
可修改性和个性化定义灵活性更好；面向对象方式性能可能更好，但修改规则通常需要改代码或发布。

第2题：煤矿安全预警系统

问题1：

- (1) f
- (2) g
- (3) h
- (4) d
- (5) b
- (6) e

数据平衡原则：

父图与子图边界上的输入/输出数据流必须与父图中对应加工的输入/输出数据流保持一致。
一个加工的输出数据必须能由该加工的输入数据直接获得或经过加工产生。

问题2：

- (1) 项目管理员
- (2) 项目经理
- (3) 项目指标数据
- (4) 项目信息
- (5) 指标参数
- (6) 事故及影响因素参数

说明：原解析标注 (4)-(6) 可互换。

问题3：

数据流图用于描述系统中数据从外部实体进入系统、经过处理、存储并输出的过程，
帮助分析系统功能和数据流向。
数据字典用于对数据流、数据存储、数据项和加工逻辑进行定义，
保证需求分析和设计阶段对数据含义理解一致。

第3题：分布式故障检测与诊断

问题1：

心跳检测：节点周期性发送健康消息，监控方根据是否连续收到心跳判断节点是否故障。
特点是实现简单、能较快定位故障节点，适合分布式系统。
超时探测：监控方发送请求或等待响应，如果在规定时间内未收到响应，则判定可能异常。
特点是依赖超时时间设置，可能受网络延迟影响。

问题2：

原解析给出：

- (1)
- (2) b、e
- (3) f
- (4)
- (5)

(6) a、d、h

(7)(8) g、c

说明：当前 PDF 文字抽取中该题表格缺失，部分空无法还原，只保留可识别答案。

问题3：

应采用数据驱动方法。

数据驱动方法通过分析系统运行监测数据进行故障诊断，常见方法包括机器学习、统计分析和信号分析。

宇航系统复杂，难以建立精确数学模型，因此不适合完全依赖基于解析模型的方法。

第4题：全国仓储货物管理系统

问题1：

实时同步方案强调强一致性，数据库更新后主动淘汰缓存，读请求再更新缓存；

为避免缓存雪崩或并发回源，需要同步控制。

异步准实时方案强调准一致性，数据库变化后通过多线程或 MQ 等方式异步更新缓存。

本题应采用异步准实时方案，因为系统要求 1 秒内响应，读多写少，对性能要求高，

实时同步在高并发下性能不可控。

问题2：

哈希分片：对 key 做 hash 后对节点数取模，定位缓存节点。优点是简单，缺点是节点增减时大量 key 失效。

一致性哈希：将节点和 key 映射到哈希环，key 顺时针寻找存储节点。节点变化时只迁移少量 key。

采用一致性哈希的原因是扩容或缩容时只需迁移少量数据，缓存命中率下降较小。

问题3：

布隆过滤器用很长的二进制向量和多个哈希函数判断元素是否在集合中。

若判断不存在，则一定不存在，可直接拦截非法查询；若判断存在，则可能存在。

优点是内存占用小、查询效率高；缺点是有误判率、不能获取元素本身、通常不支持删除。

第5题：边缘计算智能门禁系统

问题1：

MQTT 是基于客户端-服务器的发布/订阅消息传输协议，工作在 TCP/IP 协议族上，

适合低带宽、不稳定网络和硬件性能较低的远程设备，特点是轻量、简单、开放、易实现。

问题2：

(1) HTTP

(2) MQTT

(3) MQTT

(4) MQTT

(5) HTTP

(6) HTTP

(7) 端侧识别

(8) 模型训练

(9) 设备调度平台

(10) 访客注册

问题3：

数据通信：边缘侧完成识别和比对，减少回传云端的数据量，通信更快。

数据安全：原始数据尽量本地处理和加密存储，减少网络传输带来的泄露风险。

系统性能：识别在本地完成，降低云端依赖和网络时延，响应更快。

2024年5月答案

第1题：微服务与质量属性

问题1：

微服务是一种架构风格，将单体应用拆分为一组围绕业务能力构建的小服务。服务运行在独立进程中，服务之间通常通过 HTTP/JSON 等轻量通信机制协作，并可通过自动化机制独立部署。

优点：

降低单体复杂度；
服务可独立开发；
服务可独立部署；
服务可独立扩展；
技术选型更灵活。

缺点：

不是所有系统都适合拆成微服务；
部署和运维复杂度增加；
服务间远程调用可能带来性能和稳定性问题；
分布式数据一致性更难。

问题2：

质量效用树填空按题干需求归类到性能、可用性、安全性、可修改性等质量属性。

问题3：

质量属性场景六元素：
刺激源、刺激、环境、制品、响应、响应度量。

第2题：UML 时序图消息

同步消息：发送者把控制传递给接收者后等待接收者返回。
异步消息：发送者发送消息后继续执行，不等待接收者返回。
返回消息：表示从过程调用返回。

第3题：时序图与协作图选取

时序图适合强调时间顺序、消息先后、复杂交互场景的时间线。
协作图适合强调对象之间的链接、组织结构、协作关系和通信路径。

第4题：时序图组合片段

opt：可选片段，单条件分支。
alt：条件片段，多条件分支。
par：并行片段，多个并行子片段。
loop：循环片段。
break：中断片段，带监护条件的中断。

第5题：MongoDB 空间矢量与非关系数据

MongoDB 是文档型数据库，可用 GeoJSON 存储空间矢量数据，通过 geometry 字段存储几何形状，通过 coordinates 字段存储坐标。

优势：

支持文档型非结构化数据；
支持空间索引和空间查询；
扩展性好，字段可灵活增加；
适合表达丰富地理信息和复杂对象。

第6题：HDFS 冷热数据

HDFS 可根据数据访问频率设置不同存储策略，把不活跃数据迁移到低成本存储介质。

热数据访问频繁，适合 SSD；
温数据访问频率下降，适合 SAS；
冷数据访问频率低，适合大容量低成本 SATA。

2024年11月答案

第1题：云服务器故障检测

问题1：

ping/echo：监控方发送 Echo 请求并等待响应，收到响应说明网络通路正常，超时或无响应说明可能异常。
心跳：节点周期性发送心跳信号，监控方连续一段时间未收到心跳就判定节点故障或失联。

问题2：

心跳消息通常短小，网络带宽占用少；
由被监控组件主动周期性发送，减少监控方轮询压力；
适合节点多的分布式系统；
能更快发现节点失联并触发故障恢复或切换。

第2题：Cache-Aside 架构

读流程填图：

- (1) 查询缓存/读取缓存
- (2) 缓存命中，返回数据
- (3) 缓存未命中，查询数据库
- (4) 将数据库查询结果写入缓存
- (5) 返回数据库查询结果

写流程填图：

- (1) 更新数据库
- (2) 删除缓存/使缓存失效

一致性问题答案：

线程2读缓存未命中后从数据库读到旧值；
线程1更新数据库并删除缓存；
线程2随后把旧值写回缓存，导致数据库为新值、缓存为旧值。

解决方案：

消息队列异步同步；
数据库插件或 binlog 监听同步；
触发器进行缓存同步；
延迟双删；
删除失败重试；
互斥锁或分布式锁；
设置 TTL 或版本号。

第3题：ROS

问题1：

ROS 是面向机器人软件开发的开源元操作系统，提供硬件抽象、底层驱动管理、公共功能执行、进程间消息传递、程序包管理等能力。
特点包括点对点设计、多语言支持、模块化、工具库丰富、生态活跃。

问题2：

ROS2 相比 ROS1 的改进包括：
通信机制改用 DDS；

支持 Windows、macOS、RTOS 等更多平台；
实时性增强；
安全性增强；
数据类型和序列化能力增强；
节点组织更灵活，可独立进程或同进程多个节点。

问题3：

话题通信：发布/订阅的单向通信。
服务通信：请求/响应通信。
动作通信：请求/响应过程中带连续反馈。
参数服务：基于共享参数的配置和读取。

第4题：Elasticsearch 与 RESTful

问题1：

Standard：默认分词器，英文按单词切分并转小写，中文通常按单字切分。
Simple：按非字母字符切分，转小写并过滤非字母字符。
Whitespace：只按空白字符切分，不转小写，不过滤标点和数字。
Keyword：不切分，把整个输入作为一个词。

问题2：

当前 PDF 缺少架构图，无法还原 8 个空的唯一答案。
复习时按展示层、业务层、数据层、搜索服务、Elasticsearch、数据库、微信小程序、REST API 等组件判断。

问题3：

RESTful 将资源抽象为 URL，通过 GET、POST、PUT、DELETE 等 HTTP 方法操作资源。
服务端通常返回 JSON 数据。
前端负责界面展示和用户交互，后端负责业务逻辑和数据访问。
双方通过统一接口协作，实现前后端独立开发、独立维护。

2025年5月答案

第1题：在线大模型训练平台

问题1：

a：可用性
b：安全性
c：易用性
d：性能
e：安全性
f：性能
g：易用性
h：可用性
i：可测试性
j：可修改性
k：可用性
l：易用性

争议点说明：

j 写了“具备扩展能力”，容易判断为可扩展性；但题干后半句是“3天内完成新功能部署”。
软考案例题里，凡是强调新增功能、修改规则、调整界面、几天完成变更，通常按“可修改性”归类。
可扩展性更常用于增加节点、扩容硬件、提升容量、支撑更多并发或数据规模。
所以按当前解析口径，j 归为可修改性；若题目选项单独给“可扩展性”，也要结合上下文判断。

k 写的是“数据库发生故障后，自动切换到备用数据库，保证训练不中断”。
这属于故障恢复、主备切换、持续服务，是案例分析中典型的“可用性”场景。
鲁棒性/健壮性更偏向系统面对异常输入、局部扰动、边界条件或错误状态时仍能稳定运行。

如果按更细的可靠性子属性，也可以说 k 体现容错性/故障恢复能力；但考试常用答案一般归到可用性。

问题2：

- (1) 程序执行的当前状态
- (2) 解释器引擎的内部状态
- (3) 解释器引擎

问题3：

平台适合解释器风格，因为用户提交 Python 代码后，系统需要动态解析和执行代码；
解释器可将用户代码转化为可执行逻辑，并动态匹配 GPU/CPU 等计算资源；
解释器风格还便于通过插件扩展新框架或新能力，满足平台快速迭代。

第2题：医药知识图谱智能问答

问题1：

- (1) c：业务层
- (2) b：数据层
- (3) f：结构化数据
- (4) g：数据采集
- (5) j：数据清洗
- (6) h：知识获取
- (7) k：知识管理
- (8) n：意图识别
- (9) p：知识检索
- (10) o：语句解析

问题2：

- (1) Scrapy Engine/引擎
- (2) Item Pipeline/管道
- (3) Scheduler/调度器

异步 I/O：

异步 I/O 是非阻塞输入输出方式，程序发起 I/O 操作后无需等待完成，
可继续执行其他任务，并通过回调、事件或通知机制获取结果。
它能减少线程阻塞，提高高并发爬取场景的吞吐量和响应速度。

问题3：

医药知识图谱宜采用图数据库存储。
原因是医药知识涉及疾病、药品、科室、检查等实体及复杂关系，
图数据库以节点、边、属性表达语义关系，适合图遍历、关联检索和推理；
同时结构灵活，便于医药知识持续扩展。

第3题：端侧 AI 与资源池

问题1：

云端 AI：训练和推理主要在远程云端数据中心运行，终端负责采集数据和展示结果。
端侧 AI：模型推理直接在终端设备上运行，不依赖把数据传到云端处理。
端侧 AI 优势：低延迟、隐私保护、离线可用、节省带宽、降低云端成本、可靠性更好。

问题2：

- (1) b
- (2) c
- (3) a
- (4) c
- (5) b
- (6) b

(7) b

问题3：

集中式资源池：资源集中部署和统一管理，架构简单，但有单点故障风险，扩展受中心节点限制。
分布式资源池：资源分布到多个节点，去中心化，支持横向扩展和动态增减节点。
混合云资源池：整合私有云和公有云资源，通过统一平台调度、监控和治理，兼顾弹性和安全。

第4题：Redis 主从与持久化

问题1：

- (1) 从库发起同步请求
- (2) 生成 RDB 快照
- (3) 将新增写命令写入缓冲区
- (4) 发送 RDB 文件
- (5) 发送增量命令

问题2：

- (1) 主库执行写操作
- (2) 传播写命令到从库
- (3) 从库执行命令

问题3：

RDB：定期生成快照保存到磁盘。
优点是性能较高、恢复快、文件紧凑；缺点是两次快照间宕机可能丢失数据，频繁 fork 也可能带来开销。

AOF：追加记录所有写命令。
优点是数据安全性高、恢复粒度细；缺点是文件体积大、写入开销高、恢复时需回放命令，速度较慢。

第5题：区块链农产品检验流程

问题1：

数据层：负责区块数据存储、哈希、签名、加密等基础数据结构。
网络层：负责节点通信、数据传播和去中心化网络维护。
共识层：保证各节点对账本数据达成一致。
激励层：通过激励和惩罚机制维护节点参与。
合约层：支持智能合约编写与自动执行。
应用层：承载行业应用和业务场景。

问题2：

数据录入人：录入批次、产地、检测报告等基础数据，上传原始凭证到 IPFS，调用智能合约生成哈希并上链，形成存证编号。
核对人：核对已验证数据包，进行二级加密签名，触发智能合约进入审核队列。
审核人：查看全流程链上日志，复核修改记录，最终确认后触发只读或锁定状态。

问题3：

智能合约的作用：
自治执行：满足预设条件后自动执行流程，减少人工干预。
防篡改：用哈希和链上规则校验数据，发现篡改则拒绝执行。
去中心化可信协作：操作记录由多节点共同验证，增强可追溯性和防抵赖性。

2025年11月答案方向

当前 PDF 标注题目暂缺，没有完整标准答案。可按以下方向准备：

必答：软件质量属性场景。围绕 AES-256、固定 16 字节分组、数据存储和传输，重点答安全性、保密性、完整性、性能开销、密钥管理、响应度量。

选答1：精简操作系统/智能操作系统。重点答定义、特点、资源占用、实时性、安全性、智能调度。

选答2：限界上下文。重点答 DDD、统一语言、上下文边界、领域模型隔离、上下文映射。
餐饮系统可按点餐、支付、厨房制作、配送、会员营销、门店管理划分上下文。

低质量网络传输。重点答重试、超时、断点续传、压缩、缓存、消息队列、幂等、降级、边缘缓存。

主题解题思路

1. 质量属性与架构评估题

这类题几乎年年出现，是案例分析最值得优先拿分的模块。

解题步骤：

1. 先在题干中圈出带数字的约束：0.5 秒、1 秒、3 人天、15 秒、7*24、1 小时恢复。
2. 再把每条需求归入质量属性：性能、可用性、安全性、可修改性、可扩展性、可测试性、易用性。
3. 如果题目给效用树，就按“质量属性 -> 质量属性细分 -> 原文需求编号”填。
4. 如果题目要求质量属性场景，就套六元素。
5. 如果题目要求架构风格选择，先写结论，再按题目给出的维度比较。

常见关键词：

- 性能：响应时间、吞吐量、并发数、提交后几秒内完成、查询小于 1 秒。
- 可用性：宕机、故障、切换、恢复、备用站点、热备、7*24。
- 安全性：未授权访问、攻击、防护、加密、审计、隐私、防抵赖。
- 可修改性：修改规则、调整界面、新功能部署、几天天完成。
- 可扩展性：硬件扩容、横向扩展、节点增加、服务独立扩展。
- 可测试性：调试接口、诊断、日志、远程排错。
- 易用性：多语言、用户习惯、界面风格、屏幕适配。

质量属性六元素模板：

刺激源：谁触发该事件。
刺激：发生了什么事件。
环境：在什么运行条件下发生。
制品：被影响的系统、组件或资源。
响应：系统应该如何反应。
响应度量：用什么量化指标衡量响应是否达标。

答题模板：

该需求主要体现了某质量属性。刺激源为.....，刺激为.....，环境为.....，
制品为.....，系统响应为.....，响应度量为.....。

2. 架构风格选择题

高频风格包括解释器、管道-过滤器、仓库、隐式调用、面向对象、微服务、SOA。

判断口诀：

- 动态规则、脚本、DSL、代码解析、流程可配置：解释器。

- 固定顺序的数据处理流水线：管道-过滤器。
- 多个构件围绕共享数据中心协作：仓库风格。
- 事件发布、订阅、插件扩展、发送方不关心接收方：隐式调用。
- 业务对象稳定、封装对象行为、性能要求较高：面向对象。
- 服务按业务能力拆分、独立部署、独立扩展：微服务。
- 企业级服务复用、服务编排、松耦合集成：SOA。

对比题不要只背概念，要绑定场景：

本系统的核心诉求是.....。
 候选方案A在.....方面更好，但.....方面不足。
 候选方案B更符合.....，因为.....。
 因此应采用.....架构风格。

解释器风格近年很强：

- 2021 年机器学习流程拖拽。
- 2022 年促销规则动态配置。
- 2025 年大模型训练平台 Python 代码解析。

共同特征是：用户输入规则、脚本或流程，系统解释并执行，业务变化频繁，灵活性比极致性能更重要。

3. 数据管理与缓存题

这是第二强考模块，近年几乎每套都能看到。

高频知识点：

- Redis 数据类型。
- Redis 过期删除和内存淘汰。
- Redis 主从复制。
- RDB 与 AOF。
- Cache-Aside。
- MySQL 反规范化。
- Redis 与 MySQL 一致性。
- 缓存分片与一致性哈希。
- 布隆过滤器。
- MongoDB 文档模型和 GeoJSON。
- HDFS 冷热数据分层。

Redis 数据类型映射：

计数器：String + incr/decr
 对象详情：Hash
 队列/时间线：List
 标签、好友集合、共同好友：Set
 排行榜、热销排名：ZSet
 消息通知：Pub/Sub 或 Stream

Cache-Aside 标准流程：

读：先查缓存，命中直接返回；未命中查数据库；查到后写入缓存；再返回。
写：先更新数据库，再删除缓存或使缓存失效。

缓存一致性答题模板：

并发读写时可能发生时序交错。读线程缓存未命中后从数据库读到旧值，
写线程随后更新数据库并删除缓存，读线程再把旧值写回缓存，导致数据库为新值而缓存为旧值。
可采用延迟双删、删除失败重试、消息队列或 binlog 异步删除缓存、互斥锁、TTL、版本号等方案。

反规范化答题模板：

反规范化是在满足主要业务查询性能的前提下，有意增加冗余字段、派生字段、汇总表或合并表，
减少多表连接，提升查询效率。其副作用是数据冗余和一致性维护复杂。
可通过事务、触发器、应用层同步、定时校验、物化视图或消息机制维护一致性。

一致性哈希答题模板：

普通哈希通常按 $\text{hash}(\text{key}) \bmod N$ 定位节点，节点数变化会导致大量 key 重新映射。
一致性哈希将节点和 key 映射到同一个哈希环，节点变化时只影响相邻区间的数据，
可减少缓存迁移，适合缓存节点动态扩缩容。

布隆过滤器答题模板：

布隆过滤器用位数组和多个哈希函数判断元素是否可能存在。
若判断不存在，则一定不存在；若判断存在，则可能存在。
优点是空间效率高、查询快，可拦截非法请求；缺点是存在误判，且删除困难。

4. UML、DFD、E-R 与结构化分析题

这类题通常通过图填空考察，拿分关键是“从题干原文找词”。

用例图：

- 参与者是系统外部角色，不是系统内部模块。
- 用例通常直接来自功能列表。
- 管理员、用户、患者、医生、后台人员是高频参与者。

顺序图：

- 强调对象之间消息的时间先后。
- 同步消息：发送者等待返回。
- 异步消息：发送者不等待返回。
- 返回消息：调用结果返回。

协作图：

- 强调对象之间的连接关系和协作结构。
- 与顺序图语义等价，可以相互转换。

DFD：

- 外部实体：系统外部的人、组织、系统。
- 处理：动词短语，常来自功能列表。
- 数据流：名词短语，表示传递的数据。

- 数据存储：被长期保存的数据集合。

DFD 数据平衡原则：

父图中某处理的输入、输出数据流，必须与其子图边界上的输入、输出数据流保持一致。
子图可以细化内部处理，但不能凭空增加或删除与外部相连的数据流。

E-R 图：

- 实体来自业务名词。
- 联系来自动词或业务关系。
- 属性来自单据字段、表字段、业务描述。
- 派生属性是可由其他属性计算得到的属性。
- 超类实体是多个实体共有属性的抽象。

5. Web、云、边缘与物联网题

这类题经常和架构图填空、协议选择、优缺点比较绑定。

三层/Web 架构常见答法：

表示层负责界面展示和用户交互。
业务逻辑层负责业务规则、流程控制和服务编排。
数据访问层负责数据库访问、持久化和缓存访问。

RESTful 前后端分离答题模板：

RESTful 将系统资源抽象为 URL，使用 GET、POST、PUT、DELETE 等 HTTP 方法操作资源，服务端通常返回 JSON 数据。前端负责页面展示和交互，后端负责业务逻辑和数据访问，双方通过统一接口协作，从而实现前后端分离和独立开发。

TCP/UDP 判断：

- 要可靠、有序、双向通信：TCP。
- 要低开销、实时性优先、允许少量丢包：UDP。

MQTT 判断：

- 设备数量多、带宽受限、消息轻量、发布订阅、物联网：MQTT。
- 普通 Web 接口、管理后台、REST API：HTTP。

边缘计算优势：

靠近数据源处理，降低传输时延；
减少云端带宽压力；
弱网或断网时仍可本地自治；
敏感数据可本地处理，提升隐私安全；
适合实时识别、工业控制、门禁、车联网等场景。

6. 可靠性、故障检测与高可用题

常见题型包括心跳、ping/echo、超时探测、故障切换、备用系统、监控器分类。

基本概念：

ping/echo：监控方主动发送探测请求，等待回应判断连通性。
心跳：被监控方周期性发送心跳消息，监控方根据心跳是否超时判断故障。

超时探测：请求在规定时间内未返回，就认为目标可能异常。

故障切换：主节点故障后切换到备节点继续提供服务。

心跳相对 ping/echo 的资源优势：

消息短，网络开销小；
被监控方主动报活，监控方轮询压力小；
适合节点多的分布式系统；
可结合连续丢失次数降低误判。

高可用答题固定结构：

故障检测：心跳、超时、监控、日志。
故障隔离：限流、熔断、降级、隔离故障节点。
故障恢复：主备切换、重试、自动拉起、数据恢复。
故障预防：冗余部署、负载均衡、健康检查、演练。

7. 新技术场景题

新技术题的本质是“新名词 + 老问法”。不要被名词吓住。

通用答题顺序：

先定义：它是什么。
再特点：解决什么问题。
再适用：为什么适合本题场景。
再风险：有什么缺点或挑战。
最后落地：架构图、数据流、协议、存储或一致性方案。

知识图谱：

- 关键词：实体、关系、属性、三元组、图数据库、RDF、知识抽取、知识融合、知识推理、语义检索。
- 场景：智能问答、推荐、医疗、金融、知识管理。

区块链：

- 关键词：分布式账本、不可篡改、可追溯、共识机制、智能合约、非对称加密、防抵赖。
- 场景：溯源、存证、审计、多方协作。

端侧 AI：

- 关键词：本地推理、低延迟、隐私保护、弱网可用、节省带宽、功耗约束。
- 场景：智能终端、门禁、工业现场、车载设备。

ROS：

- 关键词：节点、话题、服务、动作、参数服务、发布订阅、DDS、实时性、安全性、多平台。

Elasticsearch：

- 关键词：倒排索引、分词器、全文检索、相关性排序、RESTful API、分片副本。

限界上下文：

- 关键词：领域驱动设计、上下文边界、统一语言、领域模型隔离、上下文映射。
- 若遇到餐饮系统，可按点餐、支付、厨房制作、配送、会员营销、门店管理划分上下文。

强考知识模块优先级

P0：必须先背熟

1. 质量属性分类：性能、可用性、安全性、可修改性、可扩展性、可测试性、易用性。
2. 质量属性六元素：刺激源、刺激、环境、制品、响应、响应度量。
3. 架构风格：解释器、管道-过滤器、仓库、隐式调用、面向对象、微服务、SOA。
4. Cache-Aside：读流程、写流程、数据不一致原因、解决方案。
5. Redis：数据类型、主从同步、RDB/AOF、内存淘汰、排行榜、分布式缓存。
6. UML：用例图、顺序图、协作图、消息类型。

P1：高频拿分点

1. DFD、E-R 图、数据字典、数据平衡原则。
2. MySQL 反规范化与一致性维护。
3. 缓存分片、一致性哈希、布隆过滤器。
4. RESTful、TCP/UDP、HTTP/MQTT。
5. 云平台、边缘计算、云边协同。
6. 心跳、ping/echo、超时探测、高可用。

P2：近年趋势题

1. 大模型平台与解释器风格。
2. 知识图谱与智能问答。
3. 端侧 AI 与资源池。
4. 区块链与智能合约。
5. ROS/ROS2。
6. Elasticsearch 分词器。
7. DDD 限界上下文。
8. 操作系统精简版、智能操作系统。

考场答题策略

读题顺序

1. 先看问题，不急着读说明。
2. 找题目要求：填图、解释、比较、选择、优缺点、方案原因。
3. 回到说明中圈关键词：时间指标、故障指标、安全指标、几天天、候选方案、图中上下游。
4. 对每个问题先写结论，再补理由。
5. 字数限制题控制在 3 到 5 句，不写长散文。

比较题模板

A 方案的特点是.....，优点是.....，不足是.....。
B 方案的特点是.....，优点是.....，不足是.....。
结合本系统.....的核心需求，应选择.....，因为它更能满足.....。

解释原因题模板

该问题产生的原因是.....。
在本场景中，.....会导致.....。
因此系统会出现.....结果。
可采用.....、.....、.....等方式解决。

架构图填空题模板

先看数据/请求从哪里来。
再看中间经过哪些层、组件、协议或处理。
最后看输出到哪里。
空在箭头上多半填动作或协议，空在矩形/圆柱/菱形上多半填组件、层、存储或判断节点。

两周突击建议

如果只按案例分析提分，建议顺序如下：

1. 第1轮：质量属性、六元素、架构风格，优先保证必答架构题能拿基础分。
2. 第2轮：Redis/MySQL/缓存一致性，把 Cache-Aside、反规范化、一致性哈希、布隆过滤器背成模板。
3. 第3轮：UML、DFD、E-R，重点练图填空。
4. 第4轮：Web/云/边缘/物联网协议，背 RESTful、TCP/UDP、MQTT、边缘计算优势。
5. 第5轮：新技术概念快速过一遍，按“定义、特点、适用、优缺点、落地”模板答。

最优目标不是每题都答满，而是每个主题先写出标准骨架。案例分析评分通常按点给分，结构清楚、关键词命中，比大段空泛解释更重要。